

Rapport : Projet BDD Smart Home



Tout est “Smart” - Smart Home

L3 Info TD n°2 :

Yusuf DUMLUPINAR

Aurore GIROD

Table des Matières

INTRODUCTION – QU’EST-CE QU’UNE « SMART HOME » ?	3
CAHIER DES CHARGES	4
FONCTIONNALITES	4
TECHNOLOGIES UTILISEES	5
EN RESUME.....	5
SCENARIO	6
CONCEPTION MATERIELLE	7
CAPTEURS ET ACTIONNEURS	7
ARDUINO	7
RASPBERRY PI	8
DEVELOPPEMENT LOGICIEL	9
BASE DE DONNEES.....	9
PROGRAMMATION ARDUINO	10
PASSERELLE RASPBERRY PI	10
INTERFACE WEB	10
INTERACTIONS ENTRE COUCHES	11
ARCHITECTURE GLOBALE DU PROJET	11
CONCLUSION	12
COMPETENCES ACQUISES	12
DISTRIBUTION DES TACHES.....	12
ÉVOLUTIONS POSSIBLES.....	12

Introduction – qu'est-ce qu'une « Smart Home » ?

« Si vous pensez qu'Internet a changé votre vie, détrompez-vous. L'Internet des objets est sur le point de tout changer à nouveau ! » [nous dit Brendan O'Brien](#) à propos de l'Internet des Objets, ou Internet of Things (IoT) en anglais. Et il a raison : à l'ère du digital, notre quotidien est de plus en plus entouré d'objets connectés et tendent à former un réseau dense étroitement lié à notre vie de tous les jours. Depuis des années déjà, ces objets connectés se démocratisent au plus proche de l'Homme : dans nos propres foyers. Des volets roulants au frigidaire connecté, tout un réseau interconnecté se tisse dans nos habitations, avec des outils de plus en plus poussés visant à automatiser toutes les actions qui peuvent l'être et plus encore, et nos maisons deviennent de vrais bijoux d'ingénierie. C'est là ce que l'on appelle des maisons intelligentes : « Smart Homes ».

Mais comment se développe un tel écosystème, et à quoi sert-il ? La réponse la plus simple est encore une fois la meilleure : pour se faciliter la vie. Pourquoi devoir sortir fermer des volets manuellement si une télécommande nous permet de faire de même depuis notre canapé ? On installe donc des volets roulants. Allumer et éteindre les lumières est une corvée si on a les mains pleines ? Installons donc un outil de reconnaissance vocale qui s'en occupe juste en le demandant à voix haute. Et l'on peut même utiliser la technologie pour des tâches que l'on ne sait pas faire manuellement, comme mesurer le niveau acoustique ou le niveau d'humidité d'une pièce, ou activer une alarme si un intrus est détecté dans une maison, et bien d'autres possibilités que l'on continue d'explorer au fil des innovations et des installations de chacun.

Cela ne donne-t-il pas envie de se lancer dans la conception de son propre réseau domotique ? Bien sûr que si, et c'est d'ailleurs ce que nous avons fait tout au long de ce semestre.

Cahier des Charges

Le but du projet de ce semestre est de concevoir une « Smart Home » avec un système réaliste de capteurs et d'actionneurs, contrôlés par une application via une base de données cloud.

Fonctionnalités

Nous avons réfléchi aux fonctionnalités que nous aimerions trouver dans un système de maison intelligente, et avons dégagé deux axes principaux de fonctionnalités clés : l'information et l'action. Nous avons donc choisi nos capteurs et actionneurs pour servir l'un de ces axes ou même les deux :

- Information sur la température de différentes pièces
- Information sur l'humidité de différentes pièces
- Information sur le niveau sonore d'une pièce
- Information sur l'état des lumières de la maison (allumées/éteintes)
- Information d'armement ou désarmement de l'alarme
- Action de déclenchement de l'alarme si elle est armée et détecte un mouvement anormal
- Action d'allumage ou extinction des lumières
- Action d'allumage du chauffage
- Action d'allumage d'une aide visuelle au stationnement

Le but de ces fonctionnalités est de fournir à l'habitant de la maison les informations nécessaires à son bien-être et au contrôle optimal des différents objets connectés de son domicile, ainsi que de paramétrer des actions automatiques en fonction des informations récoltées (par exemple activer le chauffage si la température descend sous un certain seuil, déclencher l'alarme si un capteur détecte du mouvement, allumer une LED dans le garage afin de savoir si l'on s'est garé à la bonne distance du mur pour ouvrir son coffre, etc.)

Pour cela nous mettons à la disposition de l'utilisateur une interface web où s'afficheront les diverses informations ainsi récoltées en temps réel grâce à une base de données en temps réel, et où se situeront divers boutons afin d'actionner les objets connectés ou les paramétrer, toujours via la base de données reliée au réseau domotique.

Nous souhaitons aussi rendre l'interface web adaptable à diverses installations, en la rendant dépendante de la structure de la base de données pour la réorganiser en fonction des capteurs et actionneurs présents dans celle-ci.

Technologies utilisées

Pour concevoir notre projet, nous avons à disposition un Arduino et un Raspberry Pi, que nous utiliserons donc pour gérer nos capteurs et actionneurs en C++ et en Python respectivement, l'Arduino pour gérer directement les actionneurs et capteurs via ses pins et le Raspberry Pi pour faire l'intermédiaire avec la base de données.

Au niveau de la base de données, nous avons utilisé une base de données Cloud afin de permettre l'accessibilité peu importe l'appareil et l'endroit, ainsi que la résilience et l'adaptabilité de ne pas être limité par une mémoire physique comme celle du Raspberry Pi. Le NoSQL s'est imposé de par l'hétérogénéité des données des capteurs et actionneurs utilisés et la praticité d'une structure hiérarchique dans la gestion de nos équipements. Une fois ces choix effectués, nous avons étudié les différentes offres disponibles et finalement suivi les recommandations du projet et choisi Firebase, qui propose une synchronisation en temps réel (Realtime Database) et une intégration au langage Python que nous utilisons très bien gérée et facilitée.

Enfin pour l'interface, nous avons considéré le développement d'une application Android, mais l'occasion nous a été donnée de nous affranchir de cette consigne. Nous avons alors étudié différentes options au vu de nos contraintes de temps et de matériel, et opté pour le framework Flask, qui s'intègre parfaitement bien en Python lui aussi permettant donc une homogénéité et simplicité de codage, et propose une rapidité d'exécution et une consommation faible de ressources qui s'avèrent être des atouts remarquables dans la manipulation d'objets connectés, couplées à une gestion et intégration de Firebase facilitées par Python, le tout en gardant une grande liberté de design de l'interface.

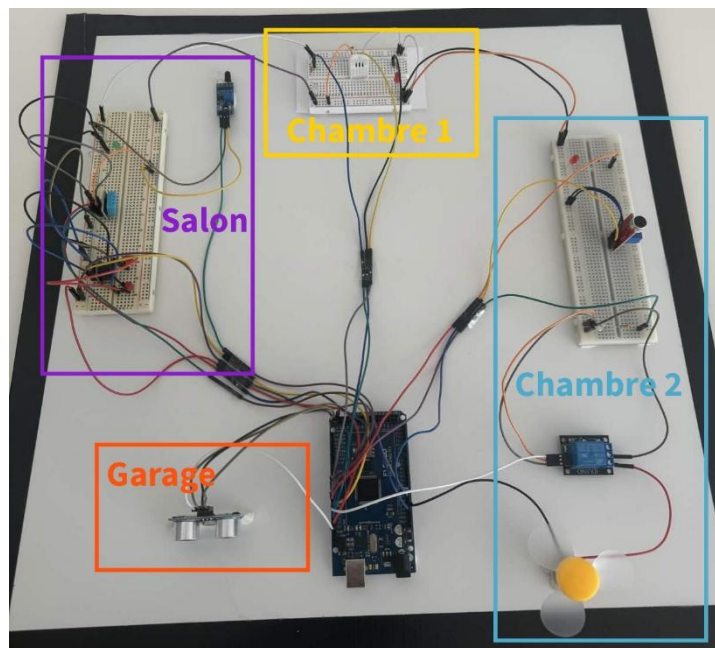
En Résumé

Nous utiliserons donc une interface Flask, une base de données Firebase Realtime Database, un Raspberry exécutant du code Python et un Arduino exécutant du code en C++ pour gérer notre réseau domotique connecté de l'action de l'utilisateur via l'interface à son exécution par les capteurs et actionneurs de notre réseau.

Scénario

Nous avons choisi comme scénario une maison composée d'un salon, de deux chambres et d'un garage, équipés comme suit :

- Salon :
 - Capteur d'humidité + de température
 - Éclairage
 - Alarme
- Chambre 1 :
 - Capteur d'humidité + de température
 - Éclairage
 - Chauffage
- Chambre 2 :
 - Capteur de niveau sonore
 - Éclairage
 - Chauffage
 - Relais et ventilateur
- Garage :
 - Capteur de distance activant une aide visuelle au stationnement



Conception matérielle

Capteurs et actionneurs

Nous utilisons divers capteurs et actionneurs dans notre réseau, ci-dessous leur liste et un lien vers un site d'explication ou une fiche technique, au besoin :

[Buzzer Iduino SE044 Module buzzer, passif \(Alarme\)](#)

[Capteur infrarouge de détection d'obstacle FC-51 \(Alarme\)](#)

[Capteur de température et d'humidité DHT11 ST052 \(Salon\)](#)

[Capteur de température et d'humidité DHT22 \(Chambre 1\)](#)

[Relais KY-019 Module relais 5V \(Chambre 1\)](#)

[Capteur acoustique Iduino 1485297 analogique \(Chambre 2\)](#)

[Capteur Ultrason HC-SR04 \(Garage\)](#)

LEDs simples

Une charge de travail importante a consisté à identifier tous les capteurs et actionneurs, trouver et étudier leurs fiches techniques associées, établir les branchements nécessaires (résistances, valeurs de résistance, branchements, alimentation, pins analogues ou digitales etc.).

Arduino

Nous avons utilisé pour notre montage un [ELEGOO Mega ATmega 2560](#), équivalent à un Arduino Mega 2560, possédant un nombre de pins digitales et analogiques suffisant pour nous éviter toute inquiétude liée au nombre de broches disponibles pour notre projet. Ses capacités de mémoire sont aussi avantageuses pour permettre le stockage de codes plus longs notamment avec des bibliothèques lourdes, toujours sans inquiétude. Ses multiples ports série permettent aussi de gérer à la fois la communication avec le Raspberry Pi et le débogage depuis notre PC et Arduino IDE par exemple. On le connecte à ceux-ci par câble USB.

Raspberry Pi

Nous utilisons un [Raspberry Pi 4 modèle B](#) mis à notre disposition comme passerelle entre le local (notre Arduino et notre montage) et le Cloud (Firebase et, derrière elle, notre application Web). Sa capacité à gérer le threading nous permet de faire tourner notre serveur Flask tout en gérant l'échange d'informations en série avec l'Arduino et la communication avec la base de données, le tout avec presque aucune latence.

L'architecture Raspberry est robuste et consomme peu, tout en fournissant une très bonne connectivité aux composants de notre réseau domotique, ce qui en fait un atout tout désigné dans notre réseau de maison connectée.

Développement logiciel

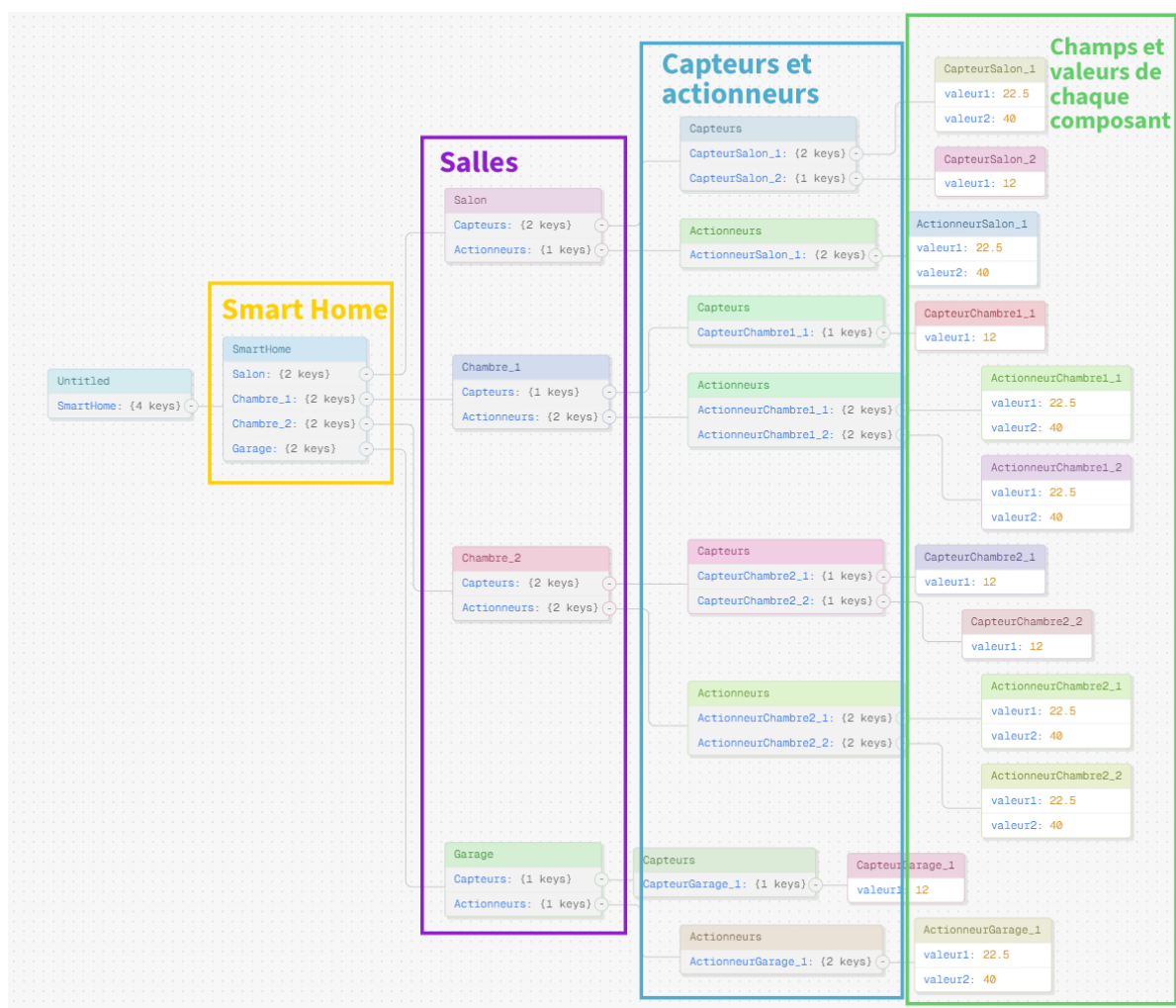
Base de données

Les données sont stockées dans Firebase en un arbre JSON.

Elles sont classées par salle, et dans chaque salle sont séparés les capteurs et les actionneurs, chacun d'entre eux ayant ses champs et valeurs de stockés.

Sa gestion est simple et ne nécessite que de mettre en place une architecture et de définir quels champs sont nécessaires à chaque capteur/actionneur.

On obtient alors un arbre JSON de cette forme :



On notera que nous n'avons qu'une Smart Home, mais que notre base de données pourrait en contenir plusieurs dans le cadre d'utilisation dans plusieurs maisons (pour plusieurs utilisateurs par exemple) et chaque Smart Home aurait donc son propre arbre de ce type, relié à la racine qui est le nœud tout à gauche dans notre schéma.

Programmation Arduino

Une fois le montage physique effectué, la programmation Arduino via [Arduino IDE](#) (C++) consiste à définir les pins de tous les capteurs, entrées et sorties, à initialiser les variables clés telles que l'état d'armement de l'alarme, puis à initialiser les fonctions nécessaires à l'exploitation des capteurs et des actionneurs avant d'enfin lancer la boucle d'actions et d'écoute à effectuer en permanence en fonction des entrées et sorties de nos pins.

On reçoit aussi en liaison en série des instructions du Raspberry Pi en fonction desquelles on effectue diverses modifications aux états de nos pins, puis envoie les informations de nos capteurs et actionneurs au Raspberry Pi en série aussi.

Passerelle Raspberry Pi

Le Raspberry Pi gère la communication entre l'Arduino et la base de données. Il se connecte en série à l'Arduino, lit les données de capteurs qu'il lui envoie, et les découpe pour en faire des valeurs utilisables qu'il peut ensuite envoyer à la base de données. De même, il récupère des données de la base de données et, s'il détecte un changement (par exemple une LED dont la valeur passe d'allumée à éteinte), il envoie en série une instruction qui sera lue par l'Arduino et changera l'état de sortie de la pin correspondante (ici celle de la LED).

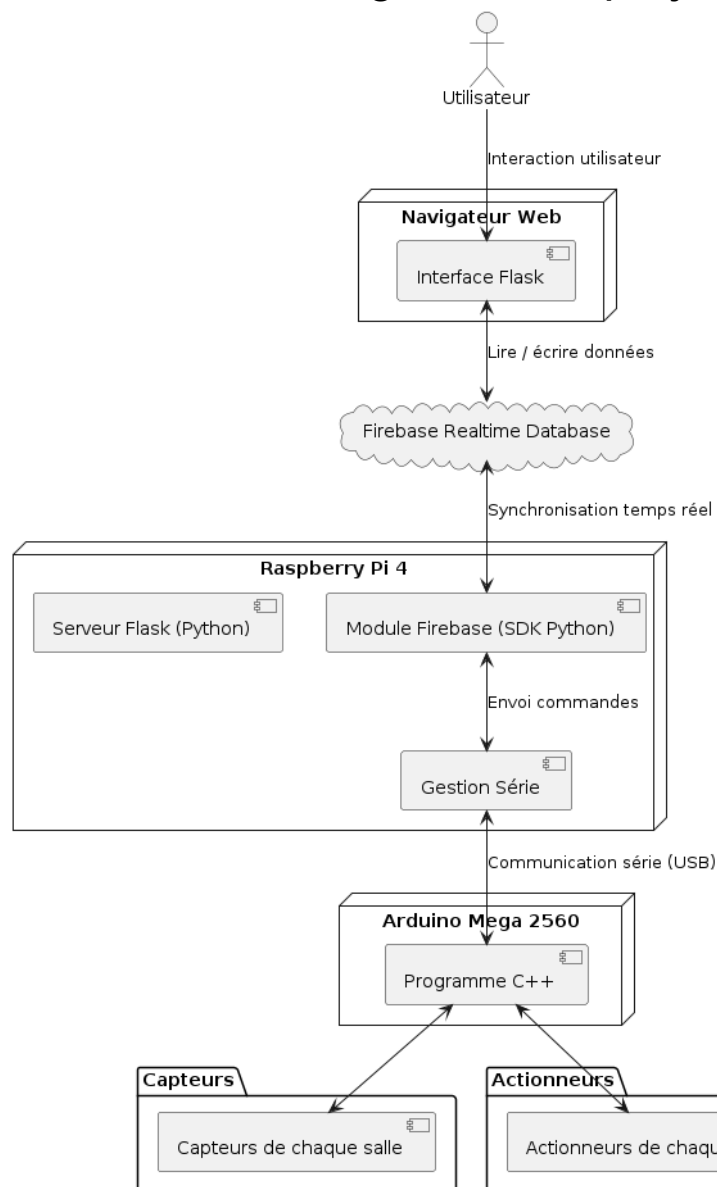
Interface Web

L'interface Web Flask est aussi gérée par le Raspberry Pi, plus précisément par du code Python s'exécutant sur celui-ci. Il exploite les données tirées de la base de données aussi, afin de les afficher sur l'interface, et met à jour la base de données en fonction des entrées de l'utilisateur, comme l'armement de l'alarme par exemple. L'utilisation de Python rend l'utilisation de la base de données facile de par la compatibilité Firebase-Python via le [SDK Python Firebase Admin](#).

Interactions entre couches

La base de données est le cœur de notre système. C'est dans la base de données que sont stockées les données des capteurs de notre montage, qui remontent en série de l'Arduino vers le Raspberry puis sont mises à jour dans la base de données. Flask récupère ensuite ces données dans la base de données pour les afficher à l'utilisateur, qui peut choisir de les modifier en actionnant des actionneurs manuellement via des boutons ou mettre en place des actions automatiques en fonction des données des capteurs. Tout cela est envoyé à la base de données dont les valeurs sont alors mises à jour, et ces valeurs sont récupérées par le Raspberry Pi qui envoie les instructions à l'Arduino en conséquence de l'état de ces données, ce qui active ou désactive des actionneurs. Puis ce cycle boucle pour faire circuler les données en continu.

Architecture globale du projet



Conclusion

Compétences acquises

Au cours de ce projet, nous avons appris à appréhender les problématiques de la gestion d'un réseau domotique, appris à gérer la technologie Arduino, que ce soit les branchements physique ou le codage en C++ de leur fonctionnement, étudié la liaison série et le contrôle de l'Arduino via Raspberry Pi afin de faire transiter les données entre une base de données Cloud NoSQL en temps réel et les capteurs et actionneurs de notre système, ladite base de données étant elle aussi une nouveauté pour nous qui n'avions jamais encore manipulé de base de données sous cette forme. Nous avons aussi gagné en maîtrise des interfaces utilisateurs et ajouté Flask aux outils qui nous sont désormais familiers dans la conception d'applications.

Distribution des tâches

L'objectif de ce projet étant de gagner en compétence dans des domaines que nous ne connaissions pas, nous avons réparti les tâches en essayant d'attribuer en priorité les sujets aux membres qui n'en avaient pas encore d'expérience. Nous avons donc attribué comme suit :

Yusuf :

- Arduino (C++)
- Raspberry Pi (Python)
- Flask (dont Python)
- Montage physique des capteurs/actionneurs

Aurore :

- Firebase
- Liaison Firebase – Raspberry Pi
- Liaison Firebase – Flask
- Rédaction du rapport

Évolutions possibles

Nous pourrions propager l'adaptativité de notre système en fonction du schéma de la base de données à tout notre système, Raspberry Pi et Arduino compris, en rajoutant notamment les pins de branchement dans la base de données et en construisant nos

codes Python et C++ à partir des données de la base de données. Dans l'état actuel des choses, l'ajout de capteurs ou actionneurs nécessite de modifier manuellement le code de notre projet.

Nous pourrions aussi, notamment en appliquant nos connaissances de cryptographie acquises ce semestre, établir un système de connexion des utilisateurs par identification stockée chiffrée dans notre base de données.